
Program Synthesis with LLMs in Blockchain Programming

June 11, 2026

Luca Sforza Roberto Di Rosa

Program testing can be used to show the presence of bugs, but never to show their absence!

Edsger W. Dijkstra

Abstract

In this work we want to present a workflow designed to synthesize solidity projects which are the most efficient(Hopefully). We successfully implemented an environment which constrains the LLM into following the specified workflow. This was done by looking at how others have approached this kind of problem and expanding on that. We used claude code's cli tool as an agentic wrapper because the implementation of hooks is heavily consistent with the documentation and it did not give us many problems as others have. From our testing we speculate that this field is promising, and that to write efficient code we need to use fine-tuned models and the more the tokens the better the final result.

1. Introduction

Blockchain programming is a niche field. In a blockchain, one can deploy Smart Contracts (which can be thought of as classes in Java), and blockchain users can interact with them to access services. Since it is a niche field, we do not expect LLMs to perfectly know the programming language used, namely Solidity. Another challenge that this field presents is security. Once a contract is deployed, there is no turning back to fix an error; the code is indelibly etched into the blockchain. Therefore, LLM hallucinations can cause immense financial damage if the contract's behavior is not verified. Consequently, we must formally

verify the behavior of the generated code, which was the methodology used in our work. We also look not only for the correctness to the specified problem but also at the most efficient implementation, in this context for efficiency we mean the amount of gas/money a contract uses when called upon.

2. State of Art

This not a new problem, others have thought about solving it. Google tried a Methodology similar to ours (Austin et al., 2021) this gave us the concept of over-generalization, overfitting of the invariants and others. But the main limitation is that they do not try to optimize the generated code which we actually do. We could not find others who tried to do the same thing as ours they usually tried something much simpler (Ye et al., 2025) (De Vito et al., 2025) which tries to fix compilation problems. Others have tried to use LLMs to search for security vulnerabilities (Le et al., 2024). Here (Chen et al., 2026) they tried to make a framework for the same purpose as ours, but the testing is lacking since they seem to use only unit testing. Some have tried to use LLMs to "verify" the generated code (Bartoletti et al., 2026) but as they admit in their paper, it's not a substitution of formal or even fuzzy testing.

3. Methodology

3.1. Our Approach to Software Engineering

In the world of software engineering we model the process of building a program as show in figure 1. We

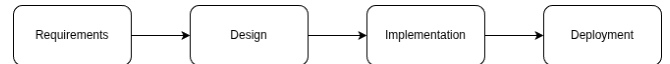


Figure 1. Software Development Cycle

aim to automate only the implementation phase, all of the other phases are handled by humans. The kind of implementation we want to automate is TDD(Test-Driven-Development), but we heavily discourage the

Email: Luca Sforza
<sforza.2050030@studenti.uniroma1.it>, Roberto Di
Rosa <dirosa.2043388@studenti.uniroma1.it>.

usage of Unit Tests. The main reason is that unit tests do not prove that the candidate program works for any state of the system i.e. for any possible input. The invariants of the system which must not be broken are defined as tests in the target language. The role of the smt solver is to prove that the invariants we defined hold for all of the possible states in the system.

$$S = \{s \mid s \text{ is a constraint in some type of logic} \} \quad (1)$$

$$A = \{\pi \mid \forall s \in S \implies \pi \models s\} \quad (2)$$

Formally in the design phase we defined a set of constraints S and A the set of all possible programs which model our system, this is called model checking. But we relaxed the definition of A permitting programs for which it is not feasible to prove their correctness for every possible state, therefore we are actually doing partial model checking.

3.2. Workflow implementation

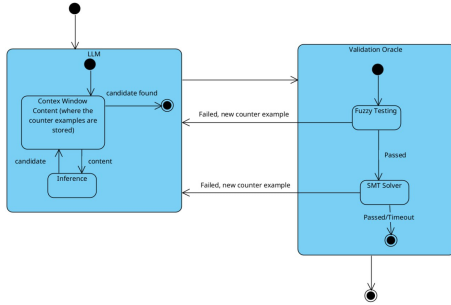


Figure 2. Synthesis loop

We implemented this idea by following the example of (David and Kroening, 2017):

1. Synthesis by sketch (David and Kroening, 2017) is done in the following manner, the Analyst/Designer formalizes the User requests¹. These formalizations are then transformed into the skeleton of the project via a **deterministic code generator**. The analyst does this to diminish the search space of all possible programs, the LLM completes the skeleton.
2. Synthesis by Counter Example (David and Kroening, 2017) as shown in this Figure 2 The LLM proposes a program which is then checked by the smt solver, if the solver finds a counter-example

¹i.e. UML,ER,FOL,SOL

the LLM continues the generator loop. We optimized this by adding a **fuzzy testing** phase which allows us to skip the SMT Solver if the fuzzy tester fails.

LLMs by nature are non deterministic ^{*2}, therefore we need to constrain them to follow the workflow. We do this by following the **Hook** and **MCP** standards. The implemented hooks are triggered at:

1. PreToolUse: Is used to make the model only edit the files we want to be modified ³.
2. PostToolUse: Is used to assure that the model does not modify the skeleton of the application⁴.
3. Stop: Is used to guarantee a final state for the model checking, so the SMT solver must either succeed or timeout otherwise the LLM must continue with the generation of candidates.

We also developed an MCP server which exposes different tools to the model which allows the model to compile, do only fuzzy testing and to propose a candidate for complete testing. The MCP tool gives back the counter-example if necessary.

3.3. Why did we choose Blockchains?

There are some technical reasons: we can easily tell which implementation is better than the others.

Let $\mathcal{D}_\pi$ gas distribution of π on fuzzy input.

$$\psi(p) = \text{Median}(\mathcal{D}_\pi) = \inf\{g \in \mathbb{N} : F_\pi(g) \geq 1/2\},$$

$$F_\pi(g) = \Pr_{X \sim \mathcal{D}_\pi} (X \leq g).$$

$$\hat{\psi}_n(p) = \text{Median}(\text{gas}(p, x_i) : i = 1, \dots, n), x_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{D}_\pi.$$

$$\text{Objective: } \pi^* = \arg \min_{\pi \in A} \hat{\psi}_n(\pi).$$

(Gas Cost Equation)

We do this by measuring the median of the **GAS** usage of the program this metric is given by fuzzy testing as seen in **Gas Cost Equation**. This formula defines a black box problem which we use since we cannot know the real ψ . To solve this problem we chose the random search algorithm to find the possible candidates which will fill our gas cost distribution function. This means that the LLM will try different approaches to see the most optimal one for the number of runs we made.

²This can vary if we change some Hyperparameter e.g. temperature

³For example we do not want the model to change the invariants, etc...

⁴e.g. Changing the signature of functions, etc...

4. Evaluation

For the evaluation, we chose the following 3 models: Qwen3.6 27B dense fine-tuned on the Solidity programming language (This was quantized by us) Qwen3.6 27B dense generalist, and DeepSeek v4 Flash 284B MoE with 13B active parameters, and we synthesized this auction designed by us.

The two Qwen models were run locally in our own execution environment using 'llama.cpp', while DeepSeek was used via API. As shown in Figure 3, the best model was the fine-tuned Qwen3.6. Surprisingly, DeepSeek (despite being the largest model) yielded poor results in synthesis. However, it was the fastest of all to complete the implementations, though this was certainly due to the fact that we used the API. On the other hand, the non-fine-tuned Qwen3.6 was the slowest.

As can be seen in the pareto Figure 4 the points on the frontier are the best implementations when we consider the amount of used tokens(input+output) and the gas which the implementation uses, this results suggest that to have good gas usage we must use more tokens. We used a System Prompt that explained the tools and the constraints we impose on the model to perform the synthesis (hooks, MCP tools, limits on executable commands, etc...).

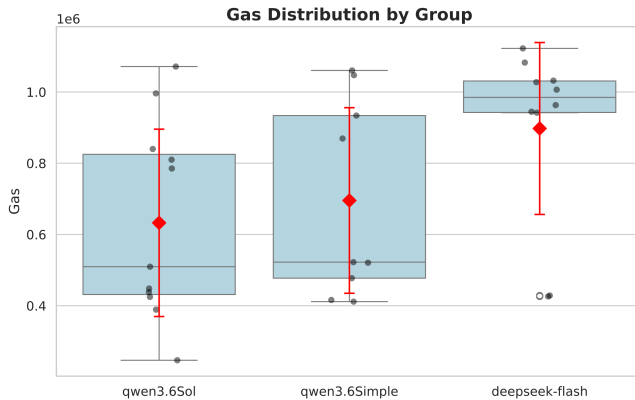


Figure 3. Box Plot of the gas usage across all implementation

5. Conclusion

5.1. Reproducibility

We wanted to make the testing Reproducible, we developed a system to do so, but we encountered an unovercomable issue. We set the seed to our inference engine and our fuzz tester, however when doing

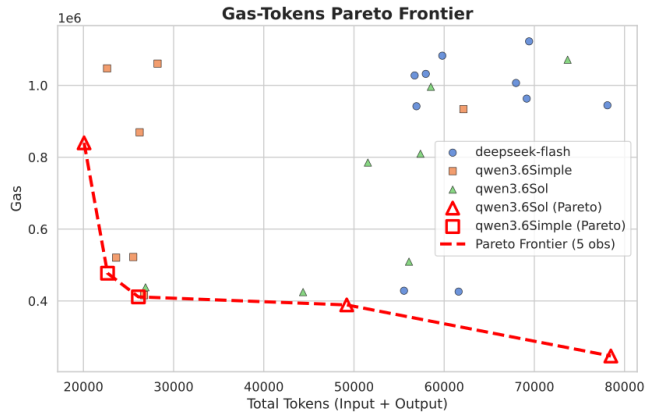


Figure 4. Pareto plot of the various implementations comparing tokens used and gas cost

multiple GPU-inference the behaviour^{5 6} is non deterministic because floating point operations are not commutative and the fact that we have multiple gpus implies that the way the operations are done cannot be always the same. Since the gpus cache will behave in a different manner each time and cannot be manipulated. We could just set the top-k to 1 but this is conceptually wrong because in this way we are doing only exploitation and no exploration of the problem, since the llm would just spit out the same output no matter the seed.

5.2. Future works

We believe that this has potential, to see if it actually has that we must test it with bigger projects like: our SSA project, eip-4626, uniswap. We actually tried to synthesize our SSA project, alas for time reasons we could not include the results, but from what we have seen it seemed to work pretty well so we remain hopeful to test this further. We also think that to synthesize big projects with hundreds of contracts we may need to use a divide et impera approach, which would make the model synthesize the contracts not all at the same time but in an ordered manner.

⁵<https://github.com/ollama/ollama/pull/5760>

⁶<https://github.com/ggml-org/llama.cpp/issues/8593>

References

- J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, Aug. 2021.
- M. Bartoletti, E. Lipparini, and L. Pompianu. Llms as verification oracles for solidity. arXiv preprint arXiv:2509.19153v2, 2026.
- W. Chen, Z. Peng, X. Yin, C. Ni, Chao e Ying, B. Xie, and Y. Luo. Solagent: A specialized multi-agent framework for solidity code generation. arXiv preprint arXiv:2601.23009v1, 2026.
- C. David and D. Kroening. Program synthesis: Challenges and opportunities. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 375(2104), 2017. doi: 10.1098/rsta.2015.0403.
- G. De Vito, D. D’Amici, F. Izzo, F. Ferrucci, and D. Di Nucci. Llm-based generation of solidity smart contracts from system requirements in natural language: the astrakode case. In *Proceedings of the 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 170–181. IEEE, 2025. doi: 10.1109/SANER64311.2025.00024.
- T.-T.-H. Le, J. Kim, S. Lee, and H. Kim. Robust vulnerability detection in solidity-based ethereum smart contracts using fine-tuned transformer encoder models. *IEEE Access*, 12:154700–154716, 2024. doi: 10.1109/ACCESS.2024.3482389.
- L. Ye, M. Li, D. Zhao, J. Sun, and X. Ren. Bridging solidity evolution gaps: An llm-enhanced approach for smart contract compilation error resolution. In *Proceedings of the 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 223–232. IEEE, 2025. doi: 10.1109/ICSME64153.2025.00029.